

What We Built This Session

NGO Dashboard · Image 403 Fix · Refresh Bug Fix · Auth Context · Protected Route

1 NGO Dashboard — View Available Donations

What We Built

Feature	Description
API Call	GET /donations/available — fetches available donations
Display	Responsive grid showing food name, quantity, expiry, pickup address, image
Accept	Accept button → PUT /donations/{id}/accept → refreshes list
Loading	Spinner while fetching, empty state when no donations

Key Functions

```
fetchDonations()
const fetchDonations = () => {
  setLoading(true);
  axiosInstance.get('/donations/available')
    .then((response) => {
      setDonations(response.data.content || []);
      setLoading(false);
    })
    .catch((error) => {
      toast.error(error.response?.data || 'Failed to load donations');
      setLoading(false);
    });
};
```

```
acceptDonation(id)
const acceptDonation = (id) => {
  axiosInstance.put(`/donations/${id}/accept`)
    .then(() => {
      toast.success('Donation accepted!');
      fetchDonations(); // Refresh list after accepting
    })
    .catch((error) => {
      toast.error(error.response?.data || 'Failed to accept donation');
    });
};
```

2 Image 403 Error — Fixed

■ Images from /uploads/ returned 403 Forbidden because Spring Security was blocking static file access.

Fix 1 — SecurityConfig.java

Add the uploads path to the permitAll() list:

```
SecurityConfig.java
.requestMatchers('/uploads/**').permitAll()
```

Fix 2 — WebConfig.java (New File)

Spring Boot doesn't serve files from uploads/ by default. Create this config to map the URL to the filesystem:

```
src/main/java/com/foodbridge/config/WebConfig.java
package com.foodbridge.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.ResourceHandlerRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void addResourceHandlers(ResourceHandlerRegistry registry) {
        registry.addResourceHandler('/uploads/**')
            .addResourceLocations('file:uploads/');
    }
}
```

✓ Two changes required: SecurityConfig (allows requests) + WebConfig (serves the files)

3 Refresh Bug — Fixed

■ On page refresh, user was redirected to /login even though a valid token existed in localStorage.

Root Cause

AuthContext initialized with **user = null**. Before the **useEffect** could read **localStorage**, **ProtectedRoute** saw null and immediately redirected to /login.

Fix — Add a loading State

```
AuthContext.jsx — Key Changes
const [loading, setLoading] = useState(true); // Start as loading
useEffect(() => {
    const jwt = localStorage.getItem('token');
    if (jwt) {
        const decoded = jwtDecode(jwt);
        setUser({ token: jwt, role: decoded.role, email: decoded.sub });
    }
    setLoading(false); // Done checking — now ProtectedRoute can decide
}, []);
```

```
ProtectedRoute.jsx — Key Changes
if (loading) {
    return <Spinner />; // Wait — don't redirect yet
}
if (!user) {
```

```

    return <Navigate to='/login' />; // Safe to redirect now
  }
}

```

4 Complete AuthContext.jsx

src/context/AuthContext.jsx

```

import { createContext, useEffect, useState } from 'react';
import { jwtDecode } from 'jwt-decode';
export const AuthContext = createContext();
export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);
  useEffect(() => {
    const jwt = localStorage.getItem('token');
    if (jwt) {
      try {
        const decoded = jwtDecode(jwt);
        setUser({
          token: jwt,
          role: decoded.role,
          email: decoded.sub,
        });
      } catch (error) {
        localStorage.removeItem('token'); // Remove invalid token
      }
    }
    setLoading(false);
  }, []);
  return (
    <AuthContext.Provider value={{ user, setUser, loading }}>
      {children}
    </AuthContext.Provider>
  );
};

```

5 Complete ProtectedRoute.jsx

src/components/ProtectedRoute.jsx

```

import { useContext } from 'react';
import { Navigate } from 'react-router-dom';
import { AuthContext } from '../context/AuthContext';
const ProtectedRoute = ({ children, allowedRole }) => {
  const { user, loading } = useContext(AuthContext);
  if (loading) {
    return (
      <div style={{ display:'flex', justifyContent:'center',
        alignItems:'center', height:'100vh',
        flexDirection:'column', gap:'16px' }}>
        <div style={{
          width:'50px', height:'50px',
          border:'6px solid #e5e7eb',
          borderTop:'6px solid #2563eb',
          borderRadius:'50%',

```

```

        animation:'spin 1s linear infinite'
      }}</div>
      <p style={{ color:'#4b5563', fontSize:'18px' }}>Loading...</p>
      <style>{`
        @keyframes spin {
          0% { transform: rotate(0deg); }
          100% { transform: rotate(360deg); }
        }
      `}</style>
    </div>
  );
}
if (!user) return <Navigate to='/login' replace />;
if (user.role !== allowedRole) return <Navigate to='/' replace />;
return children;
};
export default ProtectedRoute;

```

6 Complete NgoDashboard.jsx

src/pages/ngo/NgoDashboard.jsx

```

import { useState, useEffect } from 'react';
import axiosInstance from '../../api/axiosInstance';
import { toast } from 'react-toastify';

const NgoDashboard = () => {
  const [donations, setDonations] = useState([]);
  const [loading, setLoading] = useState(true);
  useEffect(() => { fetchDonations(); }, []);
  const fetchDonations = () => {
    setLoading(true);
    axiosInstance.get('/donations/available')
      .then((res) => { setDonations(res.data.content || []); setLoading(false); })
      .catch((err) => { toast.error(err.response?.data || 'Failed to load'); setLoading(false); });
  };
  const acceptDonation = (id) => {
    axiosInstance.put(`/donations/${id}/accept`)
      .then(() => { toast.success('Donation accepted!'); fetchDonations(); })
      .catch((err) => toast.error(err.response?.data || 'Failed to accept'));
  };
  return (
    <div className='min-h-screen bg-gray-50 py-8 px-4'>
      <div className='max-w-7xl mx-auto'>
        <h1 className='text-3xl font-bold text-gray-800'>■ Available Donations</h1>
        <p className='text-gray-500 mt-1'>Browse and accept food donations from donors</p>
        {loading ? (
          <div className='flex justify-center items-center h-64'>
            <div className='animate-spin rounded-full h-12 w-12 border-b-2 border-blue-600'></div>
          </div>
        ) : donations.length === 0 ? (
          <div className='text-center p-12'>
            <p className='text-gray-500'>■ No donations available right now</p>
          </div>
        ) : (
          <div className='grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6 mt-6'>

```

```

        {donations.map((donation) => (
            <div key={donation.id} className='bg-white rounded-xl shadow-md overflow-hidden'>
                {donation.imageUrl && (
                    <img
                        src={`http://localhost:8080/uploads/${donation.imageUrl}`}
                        alt={donation.foodName}
                        className='w-full h-48 object-cover'
                    />
                )}
                <div className='p-5'>
                    <h2 className='text-xl font-semibold'>{donation.foodName}</h2>
                    <p className='text-sm text-gray-600'>Qty: {donation.quantity} units</p>
                    <p className='text-sm text-orange-600'>Expiry: {donation.expiryTime}</p>
                    <p className='text-sm text-gray-600'>Pickup: {donation.pickupAddress}</p>
                    <button
                        onClick={() => acceptDonation(donation.id)}
                        className='mt-4 w-full bg-green-600 hover:bg-green-700 text-white
                            font-medium py-2.5 rounded-lg transition-colors'
                    >
                        Accept Donation
                    </button>
                </div>
            </div>
        ))}
    </div>
)}
</div>
</div>
);
};
export default NgoDashboard;

```

7 Key Learnings

#	Learning	Detail
1	Spring Security blocks /uploads/ by default	Add .requestMatchers('/uploads/**').permitAll() in SecurityConfig
2	Spring Boot doesn't serve files from uploads dir	Configure WebConfig.java with addResourceHandlers to map URL → filesystem
3	Refresh bug = missing loading state	AuthContext must use loading=true until localStorage is checked
4	ProtectedRoute must wait for auth check	Show a spinner when loading=true, redirect only when loading=false
5	JWT decode on init	Wrap jwtDecode() in try/catch — remove invalid tokens silently
6	fetchDonations() called after accept	Re-fetch after accept so accepted donation disappears from list

8 Git Commit

Terminal

```

git add .
git commit -m "feat: add NGO dashboard with available donations and accept functionality"

```

Files Changed This Session

File	Change	Type
SecurityConfig.java	Added /uploads/** to permitAll()	Modified
WebConfig.java	Created — maps /uploads/** to file:uploads/	New File
AuthContext.jsx	Added loading state, try/catch on jwtDecode	Modified
ProtectedRoute.jsx	Added loading check, spinner while auth loads	Modified
NgoDashboard.jsx	Built — fetch available donations, accept button	New File